

라즈베리파이 메모리 할당 결과 정리

데이터 메모리 실험

- 실험 1: 데이터 타입 최적화 + 희소 행렬 변환

- 1) Float 64 -> float 32 메모리 절약
- 2) 희소행렬 변환
- 3) 메모리 블록 단위? 전체 로드



실험1에서 사용된 데이터

- 1) 광소자 Korea_Photo_tech data-upto_1017
8/30 ~ 10/17 전체 데이터(5622,513,10)
- 2) 코에버 데이터(1374534, 114)

- 실험 2: Raspberry Pi에서 3D 배열 메모리 할당 및 주소 접근 테스트(동적 할당)

- 1) numpy로 float32 랜덤 배열 생성
- 2) ctypes를 이용하여 메모리 할당 (ctypes.Array)
- 3) ctypes 포인터를 사용하여 특정 값을 가져오기

실험1. 코드

```
def get_original_memory_size(data, name):
    if isinstance(data, pd.DataFrame) or isinstance(data, np.ndarray) and data.ndim == 2:
        # 2D array
        shape = (data.shape[0], data.shape[1])
        size_float32 = np.prod(shape) * np.dtype(np.float32).itemsize
        size_float32_MB = size_float32 / (1024 ** 2)
        print(f"✅ 희소 행렬 변환 전 크기: {size_float32_MB:.2f} MB")

        sparse_matrix = sp.csr_matrix(data.values)
        sparse_size_bytes = sparse_matrix.data.nbytes + sparse_matrix.indices.nbytes + sparse_matrix.indptr.nbytes
        sparse_size_KB = sparse_size_bytes / 1024
        sparse_size_MB = sparse_size_KB / 1024

        print(f"✅ {name} 희소 행렬 변환 후 크기: {sparse_size_MB:.2f} MB")
        return {"original_MB": size_float32_MB, "sparse_MB": sparse_size_MB}
    else:
        # 3D array
        shape = (data.shape[0], data.shape[1], data.shape[2])
        size_float32 = np.prod(shape) * np.dtype(np.float32).itemsize
        size_float32_MB = size_float32 / (1024 ** 2)
        print(f"✅ 희소 행렬 변환 전 크기: {size_float32_MB:.2f} MB")

        # 1 희소 행렬 변환 (각 2D slice마다 변환)
        sparse_matrices = [sp.csr_matrix(data[i]) for i in range(data.shape[0])]
        # 2 총 희소 행렬의 메모리 사용량 계산
        sparse_size_bytes = sum(m.data.nbytes + m.indices.nbytes + m.indptr.nbytes for m in sparse_matrices)
        sparse_size_MB = sparse_size_bytes / (1024 ** 2)

        print(f"✅ {name} 희소 행렬 변환 후 크기: {sparse_size_MB:.2f} MB")
        return {"original_MB": size_float32_MB, "sparse_MB": sparse_size_MB}

if __name__ == "__main__":
    photo_data_path = r"C:\Users\PC24-11\Desktop\Raspberry_Pi\Memory_test\photo_data_merged_array_5662_512_10.npy" # 실제 파일 경로로 변경 필요
    coever_data_path = r"C:\Users\PC24-11\Desktop\광소자\Photo_device\data\Coever_information_tech\HN.csv"

    photo_load_array = np.load(photo_data_path)
    coever_load_data = pd.read_csv(coever_data_path)

    coever_float_data = coever_load_data.select_dtypes(include=['float']).astype('float32')

    get_original_memory_size(photo_load_array, "photo_data")
    get_original_memory_size(coever_float_data, "coever_data")
```

이 코드는
희소 행렬을 사용하여 메모리가 압축 여부를 확인하
기 위함

2D array(한난-coever)와 3D array(광기술원-photo)를
따로 분리하여 코드 작성함

실험 1. 데이터 메모리 압축 실험 중 희소 행렬 적용에 관한 실험

실험 데이터

- 1) 광소자데이터(0830~1017) 사용 shape(5622,513,10)
- 2) 코에버데이터 (1374534, 114)

float 32(광소자 데이터 shape): 110.59MB

float 32(코에버 데이터 shape): 597.75MB



```
✓ 희소 행렬 변환 전 크기: 110.59 MB
✓ photo_data 희소 행렬 변환 후 크기: 232.25 MB
✓ 희소 행렬 변환 전 크기: 597.75 MB
✓ coever_data 희소 행렬 변환 후 크기: 402.51 MB
```

각 원본 데이터의 메모리 사이즈와 희소 행렬 변환 후 메모리 사이즈

광소자: 110.59MB -> **232.25MB**

코에버: 597.75MB -> **402.51MB**

실험결과

희소 행렬의 저장 방식

희소 행렬의 저장 방식은 0이 아닌 데이터와 각 데이터의 열 인덱스, 행의 시작점이 추가로 저장됨
이로 인하여 0이 많은 경우에만 메모리를 절약 할 수 있음을 알 수 있음

실험 결과 분석

광소자 데이터 의 경우는 0이 적기 때문에 메모리가 오히려 늘어 났고
코에버 데이터는 0이 많기 때문에 메모리가 절약 됨을 확인 할 수 있었음

결론

희소 행렬은 0이 많은 데이터에서만 효과적
0이 적은 데이터에서는 오히려 메모리 사용량이 증가 할 수 있음

---> 희소 행렬을 적용하기 전 0의 비율을 확인이 필요

실험 2

ctypes를 사용하여 float32 데이터를 저장할 메모리를 할당하고,
하나의 값을 가져오는 예제

순서

- numpy로 float32 랜덤 배열 생성
- ctypes를 이용하여 메모리 할당 (ctypes.Array)
- ctypes 포인터를 사용하여 특정 값을 가져오기

Raspberry Pi에서 3D 배열 메모리 할당 및 주소 접근 테스트(동적 할당)

목적:

- ctypes를 사용하여 3D Numpy 배열을 메모리에 직접 할당하고, 개별 요소의 메모리 주소를 확인
- 메모리 할당 속도 및 데이터 접근 성능 측정
- CPU 관리 정책(Governor, ondemand, powersave)에서의 실행시간 비교
 - * ondemand 사용량에 따라 클럭속도 조정(부하가 걸릴 때만 CPU 속도를 올림, 균형 잡힌 모드)
 - * powersave 400MHz 고정(CPU 속도를 최소로 고정, 전력 절약, 성능 저하)

환경: Raspberry Pi 5

메모리 크기: 1000x1000x1000(float32, 약 3.81GB)

테스트 방법:

1. 3D 배열을 ctypes로 메모리에 할당(행 우선 접근법-Row-Major Order).
2. 특정 좌표(500,500,500), (500,500,501), (500,500,502)에서 값 및 메모리 주소 확인.
3. 실행 시간을 측정하여 CPU 상태 변화에 따른 성능 비교

실험 2. 코드

```
@time_decorator
def make_dataset_memory_allocation(rows, cols, depth, seed=42):
    np.random.seed(seed)
    start = time.perf_counter()
    data = np.empty((rows, cols, depth), dtype=np.float32)

    for i in range(40): # 4GB = 100MB x 40
        data[i * 25: (i + 1) * 25] = np.random.rand(25, cols, depth).astype(np.float32)

    end = time.perf_counter()
    elapsed_time = end - start
    print(f" Execution time for data generation: {elapsed_time:.6f} microseconds")

    c_array = np.ctypeslib.as_ctypes(data.ravel())

    ptr = ctypes.cast(c_array, ctypes.POINTER(ctypes.c_float))

    base_address = ctypes.addressof(c_array)
    total_memory_size = rows * cols * depth * ctypes.sizeof(ctypes.c_float)
    last_address = base_address + total_memory_size - 1

    print(f"base address: {base_address:#x}")
    print(f"Last Address: {last_address:#x}")
    print(f"Total Memory Size: {total_memory_size / (1024**2):.2f} MB")

    return data, c_array, ptr, base_address, total_memory_size, last_address
```

기존 방식

```
ArrayType = ctypes.c_float * (rows * cols * depth)
c_array = ArrayType(*data.ravel())
```



-> 개선된 방식 `c_array = np.ctypeslib.as_ctypes(data.ravel())`

`data = np.empty((rows, cols, depth), dtype=np.float32)` -> 빈공간 확보

크기가 동일한 빈 float32 배열을 미리 생성하여 메모리 공간을 사전에 확보를 해둠

`for i in range(40)` -> 100MB 단위로 데이터 채우기

4GB 데이터를 한 번에 생성하지 않고, 100MB씩 40번에 나누어 채움(한 번에 100MB씩 처리하여 메모리 부족 OOM killed 문제 방지)

왜 25개씩 채우는가?

25개 행(rows)은(cols,depth를 1000,1000을 포함하고 있음)

그러므로 25(개) * 4(byte) * 1000,1000(cols, depth) = 100,000,000(약 100MB -> 95.367MB)

25개씩 40번을 하면 약 4GB를 채울 수 있음

--> 현재 25와 40은 대략적인 계산을 하여 사용하였고 25와 40은 다르게 사용 가능함

기존 방식은 Numpy 배열을 Python Ctypes 변환하는 방식이었음.

`ArrayType(*data.ravel())`는 NumPy 배열의 모든 요소를 하나씩 꺼내어 ctypes 배열에 복사한 방법인데
속도가 매우 느렸음

개선된 방식은 Numpy에서 제공하는 `np.ctypeslib.as_ctypes()`를 사용하여 ctypes 배열로 변환하여 사용함

Numpy 메모리를 그대로 ctypes로 사용 하여서 속도가 훨씬 빠르게 개선됨을 확인함

<https://numpy.org/doc/stable/reference/routines.ctypeslib.html>


```

@time_decorator
def get_value_and_address(ptr, base_address, x, y, z, rows, cols, depth):
    index = (x * cols * depth) + (y * depth) + z
    value = ptr[index]

    element_address = base_address + index * ctypes.sizeof(ctypes.c_float)

    print(f"Value at ({x}, {y}, {z}): {value}")
    print(f"Memory Address of ({x}, {y}, {z}): {element_address:#x}")

    return value, element_address

```

3D 배열 → 1D 배열 변환 공식

$$\text{index} = (x * \text{cols} * \text{depth}) + (y * \text{depth}) + z$$

2D 배열 → 1D 배열 변환 공식

$$\text{index} = (x * \text{cols}) + y$$

인덱스의 주소는 기준 주소 + 인덱스(1D로 변환) * c_float(크기 4) 를 곱하여 찾을 수 있음

```

if __name__ == '__main__':
    rows, cols, depth = 1000, 1000, 1000
    data, ptr, base_address, total_memory_size, last_address = make_dataset_memory_allocation(rows, cols, depth)

    x, y, z = 500, 500, 500
    get_value_and_address(ptr, base_address, x, y, z, rows, cols, depth)

    z_1 = 501
    z_2 = 502
    get_value_and_address(ptr, base_address, x, y, z_1, rows, cols, depth)
    get_value_and_address(ptr, base_address, x, y, z_2, rows, cols, depth)

    print("✅ Memory Allocation Completed")

```

base address: 0x7ffe6242c010

Last Address: 0x7fff50ade80f

Value at (500, 500, 500): 0.9038246273994446

Memory Address of (500, 500, 500): 0x7ffed996e060

get_value_and_address Execution time in: 0.000379 seconds



4byte 차이

Value at (500, 500, 501): 0.38867589831352234

Memory Address of (500, 500, 501): 0x7ffed996e064

get_value_and_address Execution time in: 0.000341 seconds



4byte 차이

Value at (500, 500, 502): 0.2411377876996994

Memory Address of (500, 500, 502): 0x7ffed996e068

get_value_and_address Execution time in: 0.000345 seconds

실험 결과 분석

(1) CPU Governor -> ondemand(**default**) mode상태

실행 횟수	데이터 생성시간(초)	데이터 메모리 할당 시간	메모리 주소 (500,500,500)	개별 값 조회 시간(초)
1	17.58	0.000341	0x7ffeedf76060	0.000022
2	17.79	0.000353	0x7ffe84576060	0.000023
3	17.62	0.000397	0x7fff0d4a6060	0.000040
4	17.56	0.000395	0x7ffe8bf66060	0.000038
5	17.82	0.000387	0x7ffeaa706060	0.000040

분석

- 데이터 생성 속도는 평균 17.76초로 빠름
- 메모리 주소는 실행마다 달라지지만 일정 패턴 유지
- 개별 값 접근 속도는 매우 빠름

(2) CPU Governor -> powersave mode (Single Core, 400MHz)

실행 횟수	데이터 생성시간(초)	데이터 메모리 할당 시간	메모리 주소 (500,500,500)	개별 값 조회 시간(초)
1	58.36	0.099854	0x7ffe78a0e060	0.000356
2	58.29	0.049679	0x7ff0764e060	0.000351
3	58.38	0.049822	0x7ffe743ae060	0.000371
4	59.56	0.049289	0x7ff1ad0e060	0.000266
5	58.93	0.049441	0x7ffea6f3e060	0.000390

분석

- CPU가 저전력 모드일 때 데이터 생성 시간이 약 3배 이상 (17->58) 증가
- 개별 값 접근 속도도 0.00035초 이상으로 증가(10배 느려짐)
- 메모리 주소 패턴은 일정하게 유지됨

결과 요약

조건	평균 데이터 생성 시간(초)	평균 데이터 메모리 할당 시간	평균 값 조회 시간 (초)	메모리 패턴
CPU ondemand	17.67	0.0003746	0.0000326	일정한 패턴
CPU powersave	58.46	0.059617	0.0003468	일정한 패턴

데이터 생성 시간을 제외하면
데이터 메모리 할당 시간과 값을 조회하는 시간은 1초 이내에 완료될 가능성이 높음.

legacy

데이터 메모리 압축 실험

- 실험 데이터

- 1) 랜덤 데이터(np.random.randn) 1000,1000,1000

- 2) 광소자 Korea_Photo_tech data-upto_1017

- 8월 30일 ~ 10월 17일 전체 데이터(만들 수 있으면 1000,1000,1000)

- 실험 1: 데이터 타입 최적화 + 희소 행렬 변환

- 1) Float 64 -> float 32 메모리 절약

- 2) 희소행렬 변환

- 3) 메모리 블록 단위? 전체 로드

- 실험 2: 데이터 반복 패턴이 있는 경우 RLE, Huffman Encoding 적용

- 1) 데이터가 반복되는지 분석(값의 중복률 확인)

- 2) RLE 적용: 연속된 값이 많다면 RLE 압축 시도

- 3) Huffman Encoding 적용: 데이터 엔트로피 기반 압축을 수행하여 메모리 절약